

Lab 3: Beginner Assembly Lab Tasks for the CPU Visual Simulator

Module	ET4047
Date	27/09/2026
Lab No.	Lab 3
Author	Luke Griffin

This lab introduces assembly programming using the CPU Visual Simulator (available at cpuvisualsimulator.github.io). Over five tasks you will see how a simple CPU uses RAM addressing, instruction flow, branching logic and basic arithmetic.

How this lab works. Each task starts with a **worked example**. It gives the goal, the complete assembly code and the expected outcome. Load the example, run it and check that you get the same result as the figure. Then complete the **Your Turn** exercises that follow. They ask you to *predict* what a program will do, *modify* the example, and then *write* a short program of your own. Write your answers in the boxes provided.

Getting to Know the Simulator

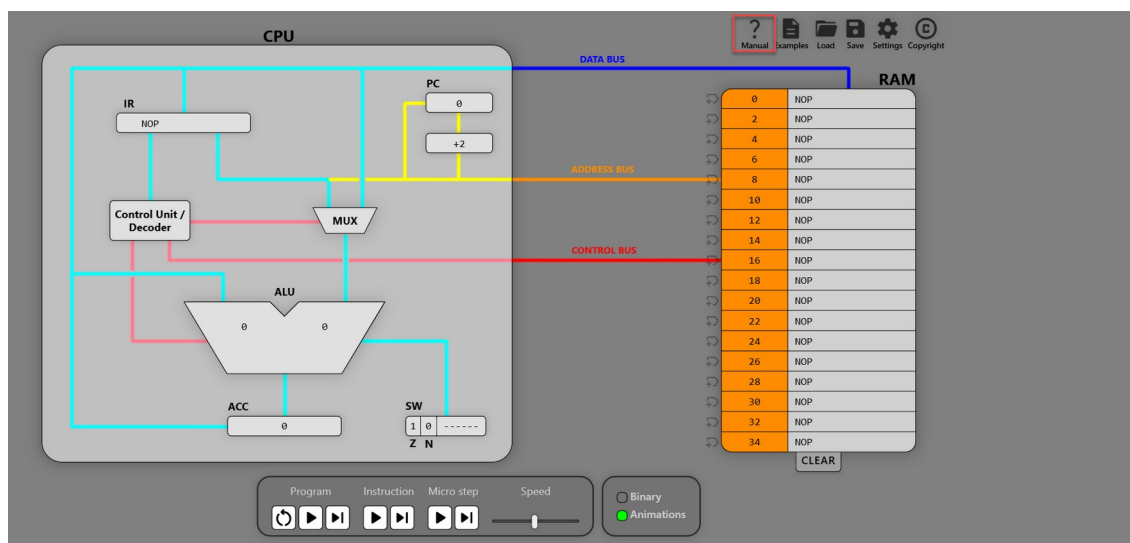


Figure 1: CPU Visual Simulator

Before starting, take a few minutes to explore the simulator. Open the **Manual** (top right) to see the instruction set and the binary encoding of each instruction. The Manual also explains how instructions are structured and includes some simple examples.

MANUAL																																							
- Introduction Instruction Set - Keyboard shortcuts - Code files - Examples	Instruction Set <table> <tr> <th colspan="2">Control Flow</th></tr> <tr> <td>NOP 00000000</td><td>No Operation; go to the next instruction</td></tr> <tr> <td>HIT 00000001</td><td>Halt execution</td></tr> <tr> <td>JMP X 00000010</td><td>Unconditional jump to the instruction at location X</td></tr> <tr> <td>JZ X 00000011</td><td>Jump on Zero: If Flag Z is set, go to instruction number X, otherwise go to the next instruction</td></tr> <tr> <td>JNZ X 00000100</td><td>Jump on Not Zero: If Flag Z is cleared, go to instruction number X, otherwise go to the next instruction</td></tr> <tr> <td>JN X 00000101</td><td>Jump on Negative: If Flag N is set, go to instruction number X, otherwise go to the next instruction</td></tr> <tr> <td>JNN X 00000110</td><td>Jump on Not Negative: If Flag N is cleared, go to instruction number X, otherwise go to the next instruction</td></tr> <tr> <th colspan="2">Data Flow</th></tr> <tr> <td>LOD X 00001001</td><td>Load content of Memory Location X (or number #X) into the Accumulator</td></tr> <tr> <td>STO X 00010000</td><td>Store the value of the Accumulator into Memory Location X</td></tr> <tr> <th colspan="2">Arithmetic Logic</th></tr> <tr> <td>ADD X 00010001</td><td>Add content of Memory Location X (or number #X) to the Accumulator. Flags are updated</td></tr> <tr> <td>SUB X 00010010</td><td>Subtract content of Memory Location X (or number #X) from the Accumulator. Flags are updated</td></tr> <tr> <td>MUL X 00010011</td><td>Multiply the Accumulator by the content of Memory Location X (or number #X). Flags are updated</td></tr> <tr> <td>DIV X 00010100</td><td>Divide the Accumulator by the content of Memory Location X (or number #X). Flags are updated</td></tr> <tr> <td>AND X 00010101</td><td>Bitwise AND between the Accumulator and the content of Memory Location X (or number #X). Flags are updated</td></tr> <tr> <td>CMP X 00010110</td><td>Subtract content of Memory Location X (or number #X) from the Accumulator. Flags are updated, but the content of the Accumulator is not modified</td></tr> <tr> <td>NOT X 00010111</td><td>Bitwise NOT of the content of Memory Location X (or number #X). The result is stored in the Accumulator</td></tr> </table>	Control Flow		NOP 00000000	No Operation; go to the next instruction	HIT 00000001	Halt execution	JMP X 00000010	Unconditional jump to the instruction at location X	JZ X 00000011	Jump on Zero: If Flag Z is set, go to instruction number X, otherwise go to the next instruction	JNZ X 00000100	Jump on Not Zero: If Flag Z is cleared, go to instruction number X, otherwise go to the next instruction	JN X 00000101	Jump on Negative: If Flag N is set, go to instruction number X, otherwise go to the next instruction	JNN X 00000110	Jump on Not Negative: If Flag N is cleared, go to instruction number X, otherwise go to the next instruction	Data Flow		LOD X 00001001	Load content of Memory Location X (or number #X) into the Accumulator	STO X 00010000	Store the value of the Accumulator into Memory Location X	Arithmetic Logic		ADD X 00010001	Add content of Memory Location X (or number #X) to the Accumulator. Flags are updated	SUB X 00010010	Subtract content of Memory Location X (or number #X) from the Accumulator. Flags are updated	MUL X 00010011	Multiply the Accumulator by the content of Memory Location X (or number #X). Flags are updated	DIV X 00010100	Divide the Accumulator by the content of Memory Location X (or number #X). Flags are updated	AND X 00010101	Bitwise AND between the Accumulator and the content of Memory Location X (or number #X). Flags are updated	CMP X 00010110	Subtract content of Memory Location X (or number #X) from the Accumulator. Flags are updated, but the content of the Accumulator is not modified	NOT X 00010111	Bitwise NOT of the content of Memory Location X (or number #X). The result is stored in the Accumulator
Control Flow																																							
NOP 00000000	No Operation; go to the next instruction																																						
HIT 00000001	Halt execution																																						
JMP X 00000010	Unconditional jump to the instruction at location X																																						
JZ X 00000011	Jump on Zero: If Flag Z is set, go to instruction number X, otherwise go to the next instruction																																						
JNZ X 00000100	Jump on Not Zero: If Flag Z is cleared, go to instruction number X, otherwise go to the next instruction																																						
JN X 00000101	Jump on Negative: If Flag N is set, go to instruction number X, otherwise go to the next instruction																																						
JNN X 00000110	Jump on Not Negative: If Flag N is cleared, go to instruction number X, otherwise go to the next instruction																																						
Data Flow																																							
LOD X 00001001	Load content of Memory Location X (or number #X) into the Accumulator																																						
STO X 00010000	Store the value of the Accumulator into Memory Location X																																						
Arithmetic Logic																																							
ADD X 00010001	Add content of Memory Location X (or number #X) to the Accumulator. Flags are updated																																						
SUB X 00010010	Subtract content of Memory Location X (or number #X) from the Accumulator. Flags are updated																																						
MUL X 00010011	Multiply the Accumulator by the content of Memory Location X (or number #X). Flags are updated																																						
DIV X 00010100	Divide the Accumulator by the content of Memory Location X (or number #X). Flags are updated																																						
AND X 00010101	Bitwise AND between the Accumulator and the content of Memory Location X (or number #X). Flags are updated																																						
CMP X 00010110	Subtract content of Memory Location X (or number #X) from the Accumulator. Flags are updated, but the content of the Accumulator is not modified																																						
NOT X 00010111	Bitwise NOT of the content of Memory Location X (or number #X). The result is stored in the Accumulator																																						

Figure 2: Instruction Set

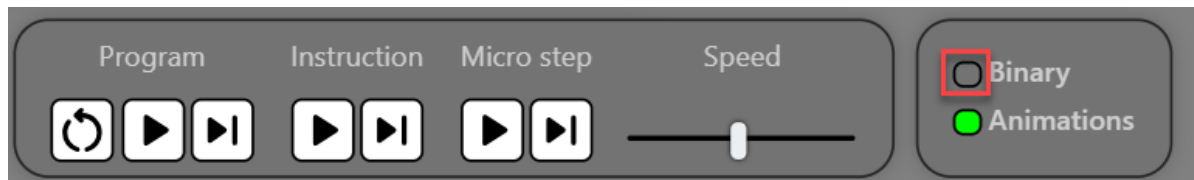


Figure 3: Control Bar

The control bar (Figure 3) lets you run a whole program, step through it one instruction or one micro-step at a time, and change the speed. The switches on the right turn the animations on and off and change the display between assembly and binary. **Tip:** step through a program one instruction at a time so you can watch the PC, ACC and RAM change.

Programs can be typed straight into RAM or written in a text editor and opened with **Load**. Figure 4 shows the file format. This example is *only* to show the layout. It is not one of the lab tasks.

```

File Edit View
ReadOneEncoder.txt ReadTwoEncoder.txt OffsetTwoEncoders.txt PID.txt PIDmotor.txt test.cpusvs test (1).cpusvs
File Edit View
      LOD X
      CMP #3
      JNZ ELSE
THEN:
      LOD #5
      ADD Y
      STO Y
      JMP ENDIF
ELSE:
      LOD #2
      ADD Z
      STO Z
ENDIF:
      LOD #8
      STO X
      HLT
X:
3
Y:
0
Z:
0

```

Figure 4: Example assembly code file format

Points to note about the format:

- A **label** (for example LOOP: or A:) names a memory address so that you can jump to it or store data there.
- # means an **immediate** value. LOD #5 loads the number 5, but LOD A loads whatever is stored at address A.
- Anything after a ; is a comment and is ignored.
- Data variables go **after** HLT so that the CPU never tries to execute them as instructions.
- Each instruction takes 2 bytes, so RAM addresses go 0, 2, 4 ... 34. That gives **18 slots** for code and data together, and the simulator places your variables at the top of RAM.

Task 1: Basic Load and Store (Memory Addressing)

Goal: Load an immediate value into the CPU, store it in RAM and then read it back from memory. This task introduces immediate and direct addressing and the basic load and store operations.

```

LOD #5      ; Load immediate constant 5 into the accumulator
STO A       ; Store the value (5) into memory location A
LOD A       ; Load the value from A back into the accumulator
HLT         ; Halt the program
A: 0        ; Variable A (will hold the stored value)
  
```

Expected Outcome: The program loads the constant 5 into the accumulator (ACC), stores it at variable A (**address 34**), then loads it back from the same address. When the program halts, address 34 contains 5 and ACC is also 5. This confirms that the value was written to RAM and read back again.

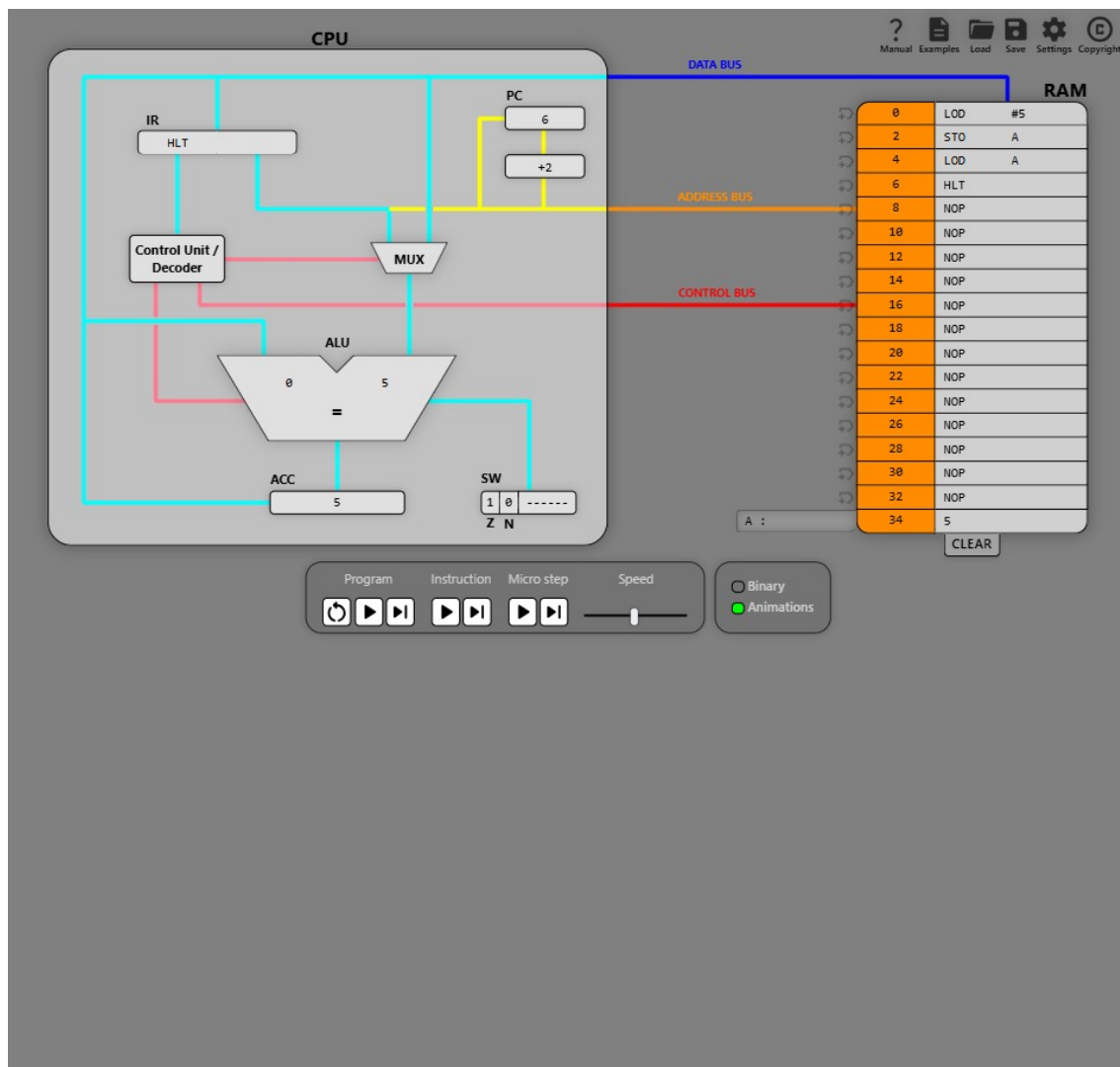


Figure 5: Basic Load and Store

Your Turn

1.1 Look at Figure 5. What value does the **PC** show when the program halts? Why does the PC go up in steps of 2 and not 1?

Your answer:

1.2 Change the program so that it also stores the value **9** in a second variable **B**. Run it and take a screenshot showing A = 5 and B = 9 in RAM.

Your code:

1.3 Write a program that **swaps** the values of two variables, starting with A: 3 and B: 8. When it halts, A should be 8 and B should be 3.

Hint: The accumulator can only hold one value at a time, so you will need a third variable to keep one value safe during the swap.

Your code:

Task 2: Adding Two Numbers from Memory (Arithmetic Operations)

Goal: Show how the CPU adds two values stored in memory. This task practises loading data from RAM, doing arithmetic in the accumulator and storing the result back in RAM.

```

LOD A      ; Load the first number (A) into the accumulator
ADD B      ; Add the second number (B) to the accumulator
STO SUM    ; Store the result into SUM
HLT        ; Halt the program
A:         3      ; First number
B:         4      ; Second number
SUM:       0      ; Will hold the result (3 + 4 = 7)

```

Expected Outcome: The program loads the first number (3) into the accumulator, adds the second number (4) and stores the result in SUM. A is at address 30, B at address 32 and SUM at **address 34**. When the program halts, address 34 contains 7 (3 + 4) and ACC is also 7.

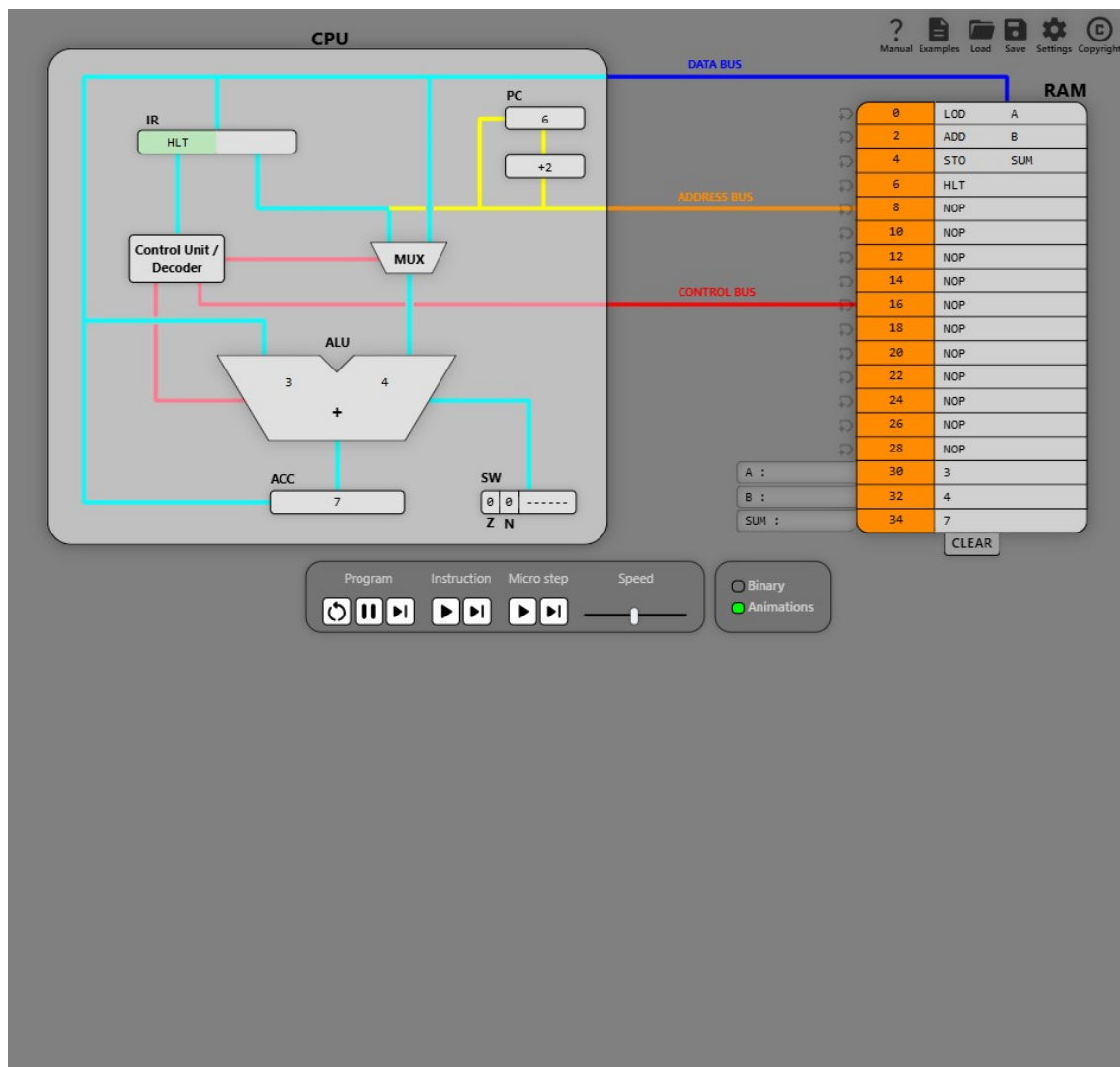


Figure 6: Arithmetic Operations

Your Turn

2.1 Change ADD B to SUB B. *Before* running it, predict the value of SUM. Then run it. What do the **Z** and **N** flags show, and why?

Your answer:

2.2 Write a program that calculates **RESULT = (A + B) × C** using A: 2, B: 3 and C: 4. Check your answer against a hand calculation.

Hint: Look up MUL in the Manual.

Your code:

2.3 Write a program that stores the **average** of A: 4, B: 7 and C: 10 in a variable AVG. Then change C to 11 and run it again. What result do you get, and why?

Your code and answer:

Task 3: Conditional Execution (Check for Zero)

Goal: Introduce conditional branching with a simple "if" scenario. The program checks a value in memory and sets FLAG depending on whether that value is zero. It shows how the CMP (compare) and JNZ (jump if not zero) instructions make a decision.

```
      LOD X      ; Load X into the accumulator
      CMP #0     ; Compare with 0 (Z flag set if X = 0)
      JNZ NOTZERO ; If X is NOT zero, jump to NOTZERO
ZERO:  LOD #1     ; X = 0: load 1 into the accumulator
      STO FLAG    ; Store 1 into FLAG (flag set)
      JMP END     ; Skip over the NOTZERO branch
NOTZERO: LOD #0   ; X != 0: load 0 into the accumulator
      STO FLAG    ; Store 0 into FLAG (flag cleared)
END:   HLT       ; Halt the program
X:     0         ; Test value
FLAG:  0         ; Becomes 1 if X = 0, otherwise 0
```

Expected Outcome: If X is **zero**, the program stores 1 in FLAG. If X is **non-zero**, it jumps to NOTZERO and stores 0 in FLAG. After running, check FLAG (**address 34**). It will be 1 if X was zero and 0 if it was not. With X = 0, as in Figure 7, FLAG ends up as 1.

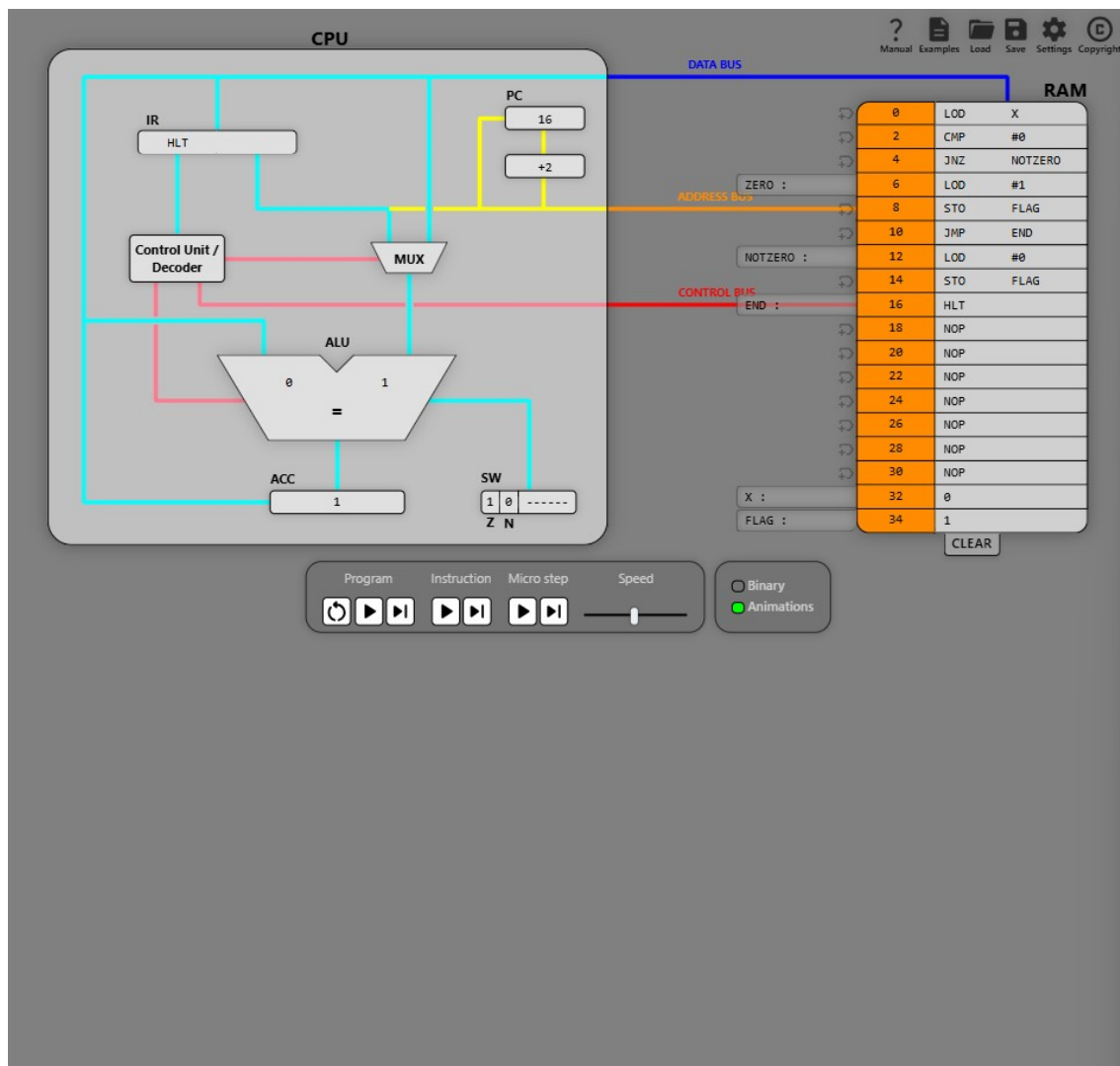


Figure 7: Check for Zero

How it works: CMP #0 subtracts 0 from the accumulator without changing it and updates the flags. The Z flag is set when the result is zero. JNZ NOTZERO jumps to NOTZERO only when Z is *not* set, which skips the code that sets FLAG to 1. If X was zero, the jump is not taken, so the CPU runs the next instructions (from the label ZERO), sets FLAG to 1 and then uses JMP END to skip the NOTZERO branch.

Your Turn

3.1 Change X to **5** and run the program again, stepping one instruction at a time. What value ends up in FLAG? Which instruction addresses were **skipped**? Take a screenshot of the final state.

Your answer:

3.2 Rewrite the program using **JZ** instead of JNZ. It must give the same FLAG value for $X = 0$ and $X = 5$.

Your code:

3.3 Write a program that sets **SMALL = 1** if X is less than 10, and **SMALL = 0** otherwise. Test it with $X = 7$, $X = 10$ and $X = 12$.

Hint: CMP #10 works out $X - 10$. If X is less than 10, the result is negative, so the N flag is set. Look up JN in the Manual.

Your code and test results:

Task 4: If-Else Logic (Comparing Two Values)

Goal: Build on Task 3 with a full if-else structure. The program compares two numbers and stores a different result depending on whether they are equal. It uses CMP, JNZ and an unconditional jump (JMP) to handle the two outcomes.

```
        LOD A          ; Load A into the accumulator
        CMP B          ; Compare the accumulator with the value at B
        JNZ NOTEQ      ; If not equal, jump to NOTEQ
        LOD #1         ; Equal: load 1
        STO RESULT     ; Store 1 into RESULT
        JMP END        ; Skip the else-part
NOTEQ:  LOD #2         ; Not equal: load 2
        STO RESULT     ; Store 2 into RESULT
END:    HLT            ; Halt the program
A:      4              ; First number
B:      4              ; Second number (try changing this)
RESULT: 0              ; 1 if equal, 2 if not
```

Expected Outcome: The program compares A (address 30) with B (address 32). If they are equal, it stores 1 in RESULT (**address 34**). If they are different, it stores 2. These numbers are just indicators: 1 means "equal" and 2 means "not equal". With A = B = 4, as in Figure 8, RESULT ends up as 1.

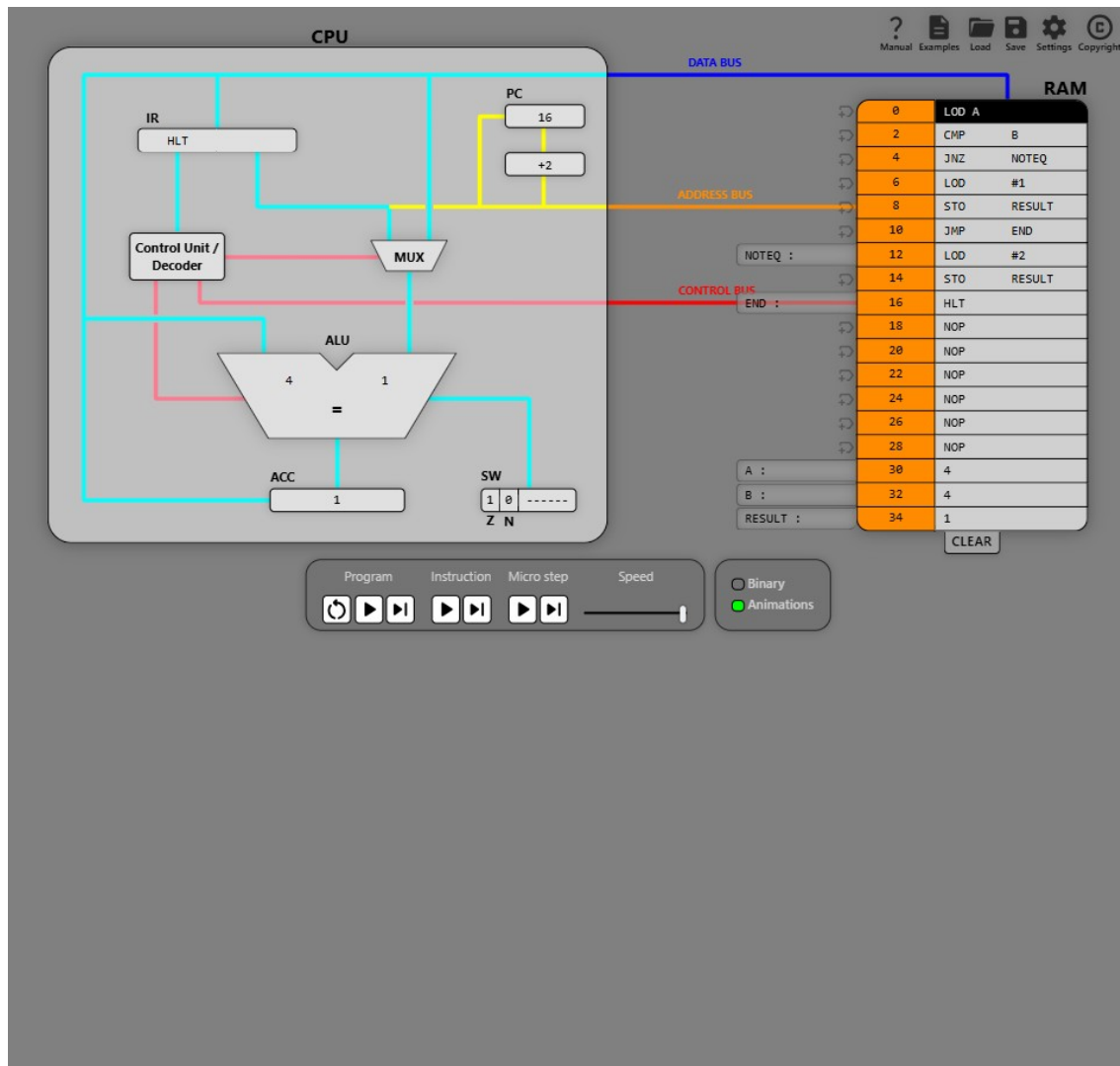


Figure 8: Comparing Two Values

Note: After running the "then" part, JMP END jumps over the "else" part, so only one of the two outcomes ever runs. The labels NOTEQ and END make the program flow easier to follow.

Your Turn

4.1 Change B to 3. Predict the value of RESULT, then run the program to check. Take a screenshot.

Your answer:

4.2 What would happen if you deleted the `JMP END` line? Try it with $A = B = 4$ and explain what you see.

Your answer:

4.3 Write a program that stores the **larger** of A and B in a variable **MAX**. Test it with $A = 4$, $B = 9$ and then with $A = 9$, $B = 4$.

Hint: After `CMP B`, the accumulator still holds A. Only the flags change. If $A - B$ is negative, B is the larger number.

Your code:

Task 5: Looping with a Countdown (Instruction Flow and Branching)

Goal: Show how a conditional jump creates a **loop**, using a simple countdown. This task shows how the program counter can jump *backwards* to repeat instructions and how the loop stops once a condition is met. It uses subtraction to decrement a counter.

```

LOD N      ; Load N into the accumulator
LOOP: SUB #1 ; Subtract 1 (decrement)
    STO N   ; Store the new value back into N
    CMP #0  ; Compare with 0
    JNZ LOOP ; If not zero, repeat the loop
    HLT     ; N reached 0: halt
N:         6 ; Starting value for the countdown
  
```

Expected Outcome: The program subtracts 1 from N each time round the loop until N reaches 0, then halts. N is stored at **address 34**. If you step through the program, you can watch the value at address 34 go down by one on each pass. When the program finishes, address 34 contains 0 and ACC is also 0.

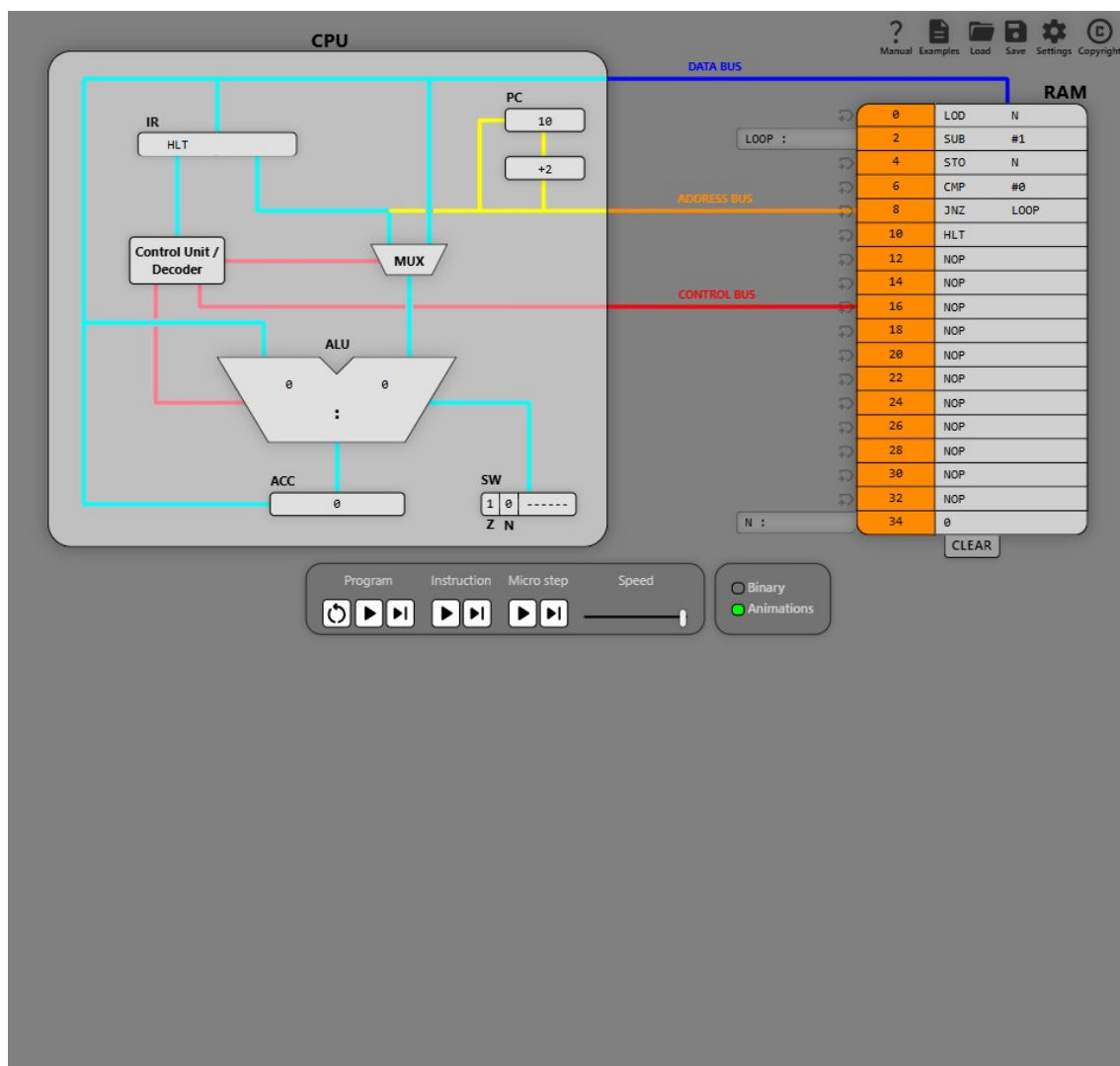


Figure 9: Instruction Flow and Branching

Your Turn

5.1 With $N = 6$, how many times does SUB #1 run? When JNZ jumps, what value is loaded into the PC?

Your answer:

5.2 Set $N: 0$ and run the program at a slow speed. What happens, and why? Then **fix** the program so that it halts straight away when $N = 0$ and still counts down correctly when $N = 6$.

Hint: Test the counter before you decrement it.

Your explanation and fixed code:

5.3 Write a program that adds up $1 + 2 + \dots + N$ and stores the total in TOTAL, using $N: 5$. The answer should be 15.

Hint: Each time round the loop, add N to TOTAL and then decrement N .

Your code:

Challenge (Optional)

C.1 Write a program that calculates the **factorial** of N ($N! = N \times (N - 1) \times \dots \times 1$) and stores it in RES. Use $N: 5$. The answer should be 120. What goes wrong if N starts at 1, and how could you fix it?

Your code and answer:

Review Questions

R1 Explain the difference between LOD #5 and LOD A.

Your answer:

R2 CMP and SUB both subtract. What is the difference between them, and why is that useful?

Your answer:

R3 Which CPU register changes when a jump is taken? Explain how this makes a loop possible.

Your answer:

R4 In your own words, what are the Z and N flags, and which instructions update them?

Your answer:

What to Submit

- This document, with your answer to every **Your Turn** exercise and review question.

- For each exercise where you write or change code: the code, and a screenshot of the simulator after the program halts.
- Your program files, saved with the **Save** button in the simulator (.cpuvs files).